

Coco: An Agentic Copilot for the Hardware–Software Co-Design Lifecycle

Samuel Kushnir^{*,2} Kavya Sreedhar^{*,1} Yeshwanth Reddy Pogula¹ Amir Yazdan Bakhsh²
 Narges Shahidi¹ Ming Liu¹ Varun Gohil³ Ravi Iyer¹
 Christina Delimitrou³ Parthasarathy Ranganathan¹ Suvinay Subramanian²

^{*}Equal contribution

¹Google ²Google DeepMind ³MIT

Abstract

Co-designing ML models and the accelerators that run them is an unusual reasoning task: architects must draw confident, high-stakes conclusions about systems that do not yet exist, and the pace of both model evolution and hardware cadence means the analysis burden grows every quarter. The evidence behind each decision—hundreds of gigabytes of fresh simulation sweeps over novel design points—is by construction absent from any LLM’s pretraining corpus. There is no external literature to retrieve; naive “chat-with-your-data” approaches hallucinate and lack reliability. We present *COCO (Copilot for Codesign)*, an agentic platform deployed with TPU architects that accelerates the co-design lifecycle of setting up experiments, sweeping simulators, and deriving insights. Coco is built as four layers: (i) a *datastore* that automatically registers every simulation sweep into a normalized relational schema, so agents ground every number in a SQL query rather than scraping heterogeneous files; (ii) a library of *tools* with typed APIs that agents compose without human orchestration; (iii) *agents* that encode recurring analysis workflows—most notably iso-execution analysis, which compares systems at matched execution configurations, including swept-but-dominated points off the Pareto frontier; and (iv) a *platform* UX whose navigation state doubles as agent context. We report early deployment experience toward a $2\times$ reduction in time-to-simulation and time-to-insight, and argue that co-design is a distinct agentic domain: its data must be retrieved rather than memorized, its workflows are recurring but context-dependent, and expert adoption hinges on UX that balances IDE-style control with interactive exploration.

1 Introduction

Hardware–software co-design for ML accelerators is a peculiar kind of performance engineering. First, it reasons about systems that do *not* yet exist: the artifact under study is a proposed chip, interconnect, or packaging option, observable only through simulation and analytical models. Second, its decisions are high-stakes and long-lived—a datacenter accelerator generation commits billions of dollars years before deployment. Third, demand is relentless and increasing: hardware cadence is accelerating, product complexity is growing, and the workload side diversifies just as fast—dense versus mixture-of-experts models [Fedus et al., 2022], proprietary versus open-source families, autoregressive versus diffusion decoding.

Concretely, the day-to-day work is dominated by a recurring question shape: *a number moved—what changed, why, and does it match our intuition?* When specifications and model assumptions are refreshed each quarter, the projected advantage of a candidate system over its predecessor can shift

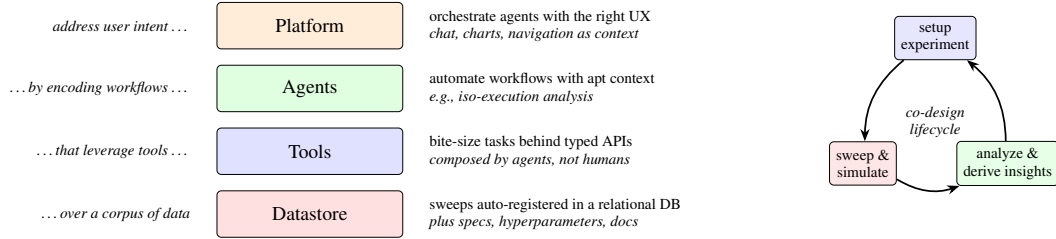


Figure 1: Coco’s four-layer stack (left) and the co-design lifecycle it accelerates (right). Each layer is consumed by the one above it; the platform’s navigation state feeds back into agent context.

by tens of percent; an architect must decompose that shift into spec changes, model changes, and sharding changes before anyone acts on it. Comparative questions have the same texture: why does a dragonfly-class interconnect benefit decode-heavy inference more than a torus—is it only interconnect pressure, or also memory bandwidth? Why is model parallelism rarely chosen on the Pareto frontier? Today, answering any of these places a senior architect on the critical path for hours-to-days of manual sweep archaeology: locating the right simulation runs, decoding configuration encodings, cross-checking simulator quirks, and assembling plots. The experts scale linearly; the questions do not.

We built COCO (*Copilot for Codesign*), an agentic platform that accelerates this lifecycle end to end—setting up experiments, sweeping and simulating, and analyzing the results into insights (Figure 1, right)—with a near-term goal of halving time-to-simulation and time-to-insight, and a longer-term goal of letting the underlying LLM itself iterate on and explore new architectural ideas. This paper describes the thesis behind Coco (§2), its four-layer architecture (§3), early deployment experience with TPU [Jouppi et al., 2023] architects (§4), and two end-to-end case studies on public TPU generations (§5).

2 Why co-design needs its own copilot

Our thesis is that co-design has unique *workflows*, *context*, and *artifacts*, and that a generic coding or chat assistant serves none of them well.

The data cannot be memorized. Every design cycle regenerates hundreds of gigabytes of simulation data over design points that are novel by construction—new chips, new topologies, new model variants. No pretrained model has seen these numbers, and unlike open-domain retrieval-augmented generation [Lewis et al., 2020] there is no external literature to fall back on: the corpus is the organization’s own sweeps, specs, and documents, and it changes under the agent’s feet. Grounding must therefore be *structural*: every quantitative claim should be traceable to a series of queries and tool calls which a human can easily validate.

The workflows recur, but the context varies. Analyses repeat in skeleton but every instance differs in the particulars. Almost every workflow consists of running a sweep, extracting Pareto frontiers, comparing systems, and decomposing a delta but the particulars: which use case, which sequence-length regime, which metric, are constantly changing. The productive division of labor is to standardize the skeleton as tools and encoded workflows and let the LLM handle the *difference*: “rerun this sweep with only the HBM bandwidth changed,” “pull peak memory usage for the Pareto-optimal points of that chart.”

The artifacts are collaborative objects. The unit of progress is not a code diff but a Pareto curve, a per-op breakdown, a utilization timeline, a comparison report. Experts need to create, navigate, diff, and share these rapidly—and the trail of artifacts a user has visited is itself the best statement of their intent.

3 The Coco stack

Datastore: sweeps as a database, not files. Every simulation sweep—serving, training, and sampling/thinking workloads, from both analytical and event-driven simulators—is automatically

registered into a normalized relational schema: test plans, workloads, systems, execution strategies, Pareto frontiers, and per-phase (prefill/decode/combined) serving points carrying latency and throughput metrics down to time-to-first-token. Agents retrieve data by constructing SQL against this schema rather than scraping CSV, proto, or JSON artifacts. In our experience this is the single largest lever against hallucination: the model never “remembers” a number, it fetches one, and the query itself is an auditable provenance record. The datastore also collates the canonical context around the numbers—system specifications, model hyperparameters, ISA and product documentation—so retrieval spans both measurements and their meaning.

Tools: bite-size tasks behind typed APIs. On top of the datastore sits a library of tools for well-specified tasks, registered with documentation and made available to LLM agents in the style of modern tool-use systems [Schick et al., 2023, Yao et al., 2023]. A representative retrieval chain—find valid system/model combinations → find valid Pareto curves → fetch a curve → fetch a point → fetch its detailed breakdown—is composed by the agent, interleaved with small amounts of inline code, with no human orchestration or bespoke glue per analysis.

Agents: encoded workflows with the right context. Agents automate the recurring workflows: run an analysis across systems and models, synthesize the per-system results into a coherent comparison, and generate a report. The flagship is *iso-execution analysis*, an analysis workflow in which we focus on a given latency target, and directly compare the two systems performance in-depth at that latency target. This includes diving into op-by-op breakdowns and pinning down exactly which operators and under which use cases the systems differ the most on performance. Architects constantly ask why an under-performing system did not adopt the winning system’s configuration (“why does the torus stop at 16-way expert parallelism when the dragonfly chooses 64-way?”). The answer usually exists in the sweep—the configuration was evaluated and dominated by a better execution strategy so it doesn’t appear on the Pareto curve. Exhuming it by hand means reconstructing equivalent configuration keys across systems and locating which shard of which sweep job produced the point. The agent retrieves the iso-sharding, iso-batch counterpart directly from the datastore and reports *why* it lost, turning a multi-hour human search into part of every report. Coco is now a multi-agent system, where the orchestrator agent can autonomously launch sub-agents with access to the same tooling. In the iso-execution analysis workflow, sub-agents are used to parallelize the analysis across different latency targets and reviewer agents are then used to check over the integrity of the analysis. The orchestrator agent then synthesizes everything to a single report for the human engineer.

Platform: UX as context capture. The platform provides the interaction grammar of architectural analysis—hover a Pareto point for details, click through to its breakdown, open the utilization timeline, diff two breakdowns across systems or execution strategies—with an embedded conversational side panel. Critically, the UX is also how agent context gets captured: the chart the user is looking at, the use case and sequence-length regime it encodes, and the trajectory of questions that led here are all attached to the user query automatically, so a user can ask “why does this point have poor utilization?” and the agent will have full understanding of the pareto curve and point in question.

4 Early experience

Coco is deployed with TPU architects and in daily use for serving-focused analyses; training and further simulator backends are being onboarded. Three observations from early use:

Structured retrieval changes trust. Analyses that quote numbers with their generating SQL are treated as grounded evidence by architects, in a way free-form LLM summaries never were. The schema, not the model, carries the burden of being right about data.

Iso-execution defaults surface real mechanisms. Comparisons that previously ended at “system A wins by $X\%$ ” now routinely end at *which* configurations each system chose and what dominated the alternatives—the level at which architects can connect results to intuition, or flag that a simulator assumption (not the hardware) drove the delta.

Context capture is half the product. The single biggest usability gap between a generic chatbot and Coco is not model quality but that Coco knows what the user is looking at. Encoding the navigation trail as agent context eliminated most of the prompt-writing that made earlier assistants feel slower than doing the analysis by hand.

Our headline target is a $2\times$ reduction in time-to-simulation and time-to-insight.

5 Case studies

Two recent analyses exercise the stack end to end. Both involve public TPU generations: the deployed Ironwood systems and the announced eighth-generation inference chip, TPU 8i [Google, 2026].

5.1 Generation-over-generation analysis: Ironwood to TPU 8i

We tasked Coco with a comparative report between a baseline Ironwood [Google Cloud, 2025] system and a (then) next-generation TPU 8i configuration, asking specifically how the increased on-chip SRAM and compute change execution. Coco synthesized the simulation sweeps into a causal, regime-scoped insight rather than a headline ratio: tripling the scratchpad (128 MB to 384 MB) specifically benefits MoE models with fewer than 64 experts under a sub-2-second latency constraint. It also explained *why*: at those model shapes, the larger SRAM admits a batch-size combination at which expert tensors can be pinned and prefetched to fit the new capacity exactly, taking HBM refetch off the critical path—and outside that regime (more experts, or looser latency targets) the benefit washes out. Every quoted number in the report links to its generating SQL.

5.2 Network topology what-if: torus versus Boardfly

Ironwood pods connect chips in a torus; TPU 8i moves to a hierarchical *Boardfly* topology—fully connected four-chip boards, fully connected eight-board groups, 36 groups per pod [Google, 2026]. We used Coco for a what-if analysis of the two interconnects under highly sharded MoE serving. Orchestrating the iso-execution tool chain, the agent held sharding and batch size fixed across topologies to isolate the network’s contribution, showing how Boardfly reshapes the all-to-all dispatch and combine latencies that dominate expert-parallel execution. It then generated side-by-side timeline breakdowns from which the architect could visually verify the bottleneck shifting from network-bound to compute-bound execution—the level of evidence at which a topology discussion stops being a matter of taste.

6 Conclusion

Hardware–software co-design requires more than generic language models; architects need tools capable of reasoning over fresh simulation data and executing complex, context-dependent workflows. Coco addresses this gap by shifting the burden of data archaeology and routine analysis from the expert to an agentic platform. Rather than relying on model memorization, Coco grounds its insights in a relational datastore and automates recurring tasks like iso-execution comparisons, all within an interface built for active architectural exploration.

While prior work has demonstrated the value of machine learning in physical design [Mirhoseini et al., 2021], domain-specific chat assistants [Liu et al., 2023], and early co-design productivity studies [Sreedhar et al., 2025], Coco elevates this assistance directly to the system-architecture level where the highest-stakes decisions are made. By significantly accelerating time-to-insight for architects today, this four-layer stack provides a practical foundation for the autonomous co-designers of tomorrow.

References

- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Google. Our eighth generation TPUs: two chips for the agentic era. <https://blog.google/innovation-and-ai/infrastructure-and-cloud/google-cloud/eighth-generation-tpu-agentic-era/>, 2026.
- Google Cloud. Ironwood: The first Google TPU for the age of inference. <https://blog.google/products/google-cloud/ironwood-tpu-age-of-inference/>, 2025.

- Norman P Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–14, 2023.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33: 9459–9474, 2020.
- Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Bayraktaroglu, et al. ChipNeMo: Domain-adapted LLMs for chip design. *arXiv preprint arXiv:2311.00176*, 2023.
- Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, et al. A graph placement methodology for fast chip design. *Nature*, 594(7862):207–212, 2021.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.
- Kavya Sreedhar, Josh Ogbonda, Pengqi Yin, Narges Shahidi, Kanthi Nagaraj, Zhijie Deng, Rami Cohen, Ton Kalker, Sameer Kumar, Amir Yazdanbakhsh, and Suvinay Subramanian. Leveraging LLMs to improve hardware-software co-design workflow productivity and accessibility. In *Workshop on Machine Learning for Computer Architecture and Systems (MLArchSys) at ISCA*, 2025.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.